

Administrative Guide

Author: Raja SP, AWS Labs

Source: <https://github.com/awslabs/aidlc-workflows>

Adaptation: Ricardo de Luna Galdino, EngSoft Learn

Download: [administrative-guide.pdf](#)

This guide documents the CI/CD infrastructure, GitHub Workflows, protected environments, secrets, variables, permissions, and release process for the `awslabs/aidlc-workflows` repository.

Audience: Repository administrators, maintainers, and AI coding agents working on this repository.

Related documentation:

- [Developer's Guide](#) — Running builds locally (CodeBuild + `act`)
- [Contributing Guidelines](#) — Contribution process and conventions
- [README](#) — User-facing setup and usage

Table of Contents

- [Repository Overview](#)
- [CI/CD Architecture](#)
- [Workflow Reference](#)
- [Release PR Workflow](#)
- [Tag Release Workflow](#)
- [CodeBuild Workflow](#)
- [Release Workflow](#)
- [Pull Request Validation Workflow](#)
- [Security Scanners Workflow](#)
- [Protected Environments](#)
- [Secrets and Variables](#)
- [Permissions Model](#)

- [Security Posture](#)
 - [Security Finding Requirements](#)
 - [Code Ownership](#)
 - [Release Process](#)
 - [Changelog Configuration](#)
 - [Updating Pinned Versions](#)
-

Repository Overview

This repository publishes the **AI-DLC (AI-Driven Development Life Cycle)** methodology as a set of markdown rule files under `aidlc-rules/`. The CI/CD infrastructure handles:

- **Continuous integration** via AWS CodeBuild (evaluation and reporting)
- **Release distribution** via GitHub Releases (zipped rule files)
- **Changelog generation** via git-cliff (changelog-first: updated before release, included in the tagged commit)

```
awslabs/aidlc-workflows/
├── .github/
│   ├── CODEOWNERS
│   ├── ISSUE_TEMPLATE/           # Bug, feature, RFC, docs templates
│   ├── labeler.yml               # Auto-label rules (path → label mapping)
│   ├── pull_request_template.md  # PR template with contributor statement
│   └── workflows/
│       ├── codebuild.yml         # CI via AWS CodeBuild
│       ├── pull-request-lint.yml  # PR validation (title, labels, merge gates)
│       ├── release.yml           # GitHub Release on tag push
│       ├── release-pr.yml        # Changelog PR before release
│       ├── security-scanners.yml # Security scanning suite (6 scanners)
│       └── tag-on-merge.yml       # Auto-tag on release PR merge
├── .claude/
│   └── settings.json             # Shared Claude Code project settings
├── aidlc-rules/                  # The distributable product
│   ├── aws-aidlc-rules/         # Core workflow rules
│   └── aws-aidlc-rule-details/   # Detailed rules by phase
├── cliff.toml                    # git-cliff changelog configuration
├── docs/
│   ├── ADMINISTRATIVE_GUIDE.md  # This file
│   └── DEVELOPERS_GUIDE.md      # Local build instructions
└── scripts/
    └── aidlc-evaluator/          # Evaluation framework (in development)
```

CI/CD Architecture

Six workflows form two distinct pipelines, a security scanning suite, plus a pull request validation gate:

Pipeline 1: Release (changelog-first)

```
flowchart TD
    A["workflow_dispatch\n(optional version input)"] --> B["release-pr.yml"]
    B --> C["Determine version\n(input or git-cliff)"]
    C --> D["Generate CHANGELOG.md\nwith git-cliff"]
    D --> E["Open PR: release/vX.Y.Z\nwith updated CHANGELOG"]

    E --> F["Human reviews\nand merges PR"]

    F --> G["tag-on-merge.yml"]
    G --> H["Extract version from\nbranch name"]
    H --> I["Create tag vX.Y.Z\nnon merge commit SHA"]

    I --> J["Dispatch release.yml"]
    J --> K["release.yml\ncreates draft release\nwith rules zip"]
    K --> L["Dispatch codebuild.yml\nafter draft exists"]
    L --> M["Manual approval\n(codebuild environment)"]
    M --> N["Run AWS CodeBuild\nupload artifacts to draft"]

    K --> O["Human reviews\nand publishes draft"]
    N --> O

    P["workflow_dispatch\n(select tag in UI)"] -.->|"manual backup\ntrigger"| M
```

The release flow is **changelog-first**: the CHANGELOG is updated *before* the tag is created, so the tagged commit always contains its own changelog entry. The flow has three human touchpoints:

1. **Merge the release PR** — reviews the changelog, triggers automatic tagging
2. **Approve the CodeBuild environment** — gates access to AWS credentials for the build
3. **Publish the draft release** — reviews artifacts, makes the release public

`tag-on-merge.yml` explicitly dispatches `release.yml` and `codebuild.yml` via `gh workflow run --ref vX.Y.Z` after creating the tag. The dispatches are **sequential**: `release.yml` runs first and is watched to completion so that the draft release exists before `codebuild.yml` uploads artifacts. This is necessary because tags created with `GITHUB_TOKEN` do not trigger `on: push: tags` events — but `workflow_dispatch` is exempt from this limitation. Both workflows also

retain `push: tags: v*` as a fallback for manual tag pushes. The `codebuild.yml` workflow requires **manual approval** via the `codebuild` protected environment before the build proceeds. The upload step handles all release states resiliently:

- **Draft exists** (normal case) — `release.yml` finishes in ~30s creating the draft; CodeBuild takes minutes, so the draft is ready when artifacts are uploaded
- **No release yet** (codebuild finished first) — creates a draft with build artifacts; `release.yml` will update it later
- **Already published** (re-run) — attempts to replace artifacts, warns gracefully if immutable

Backup strategy: If the tag-triggered CodeBuild run fails or is blocked, an admin can manually dispatch the workflow via `workflow_dispatch` and select the `v*` tag in the GitHub UI branch/tag selector. Since `github.ref` resolves to the selected tag, the upload step activates automatically.

Pipeline 2: Continuous Integration

```
flowchart LR
    A["git push main"] --> B["Manual approval<br/>(codebuild environment)"]
    C["workflow_dispatch<br/>(no tag input)"] --> B
    D["pull_request<br/>(aidlc-rules/** changed)"] --> E["rules<br/>label?"]
    E -->|yes| F["label-cleanup<br/>(remove reminder comment)"]
    F --> B
    E -->|no| I["label-reminder<br/>(warning + PR comment)"]
    B --> G["Run AWS CodeBuild"]
    G --> H["Upload workflow artifacts"]
```

Pipeline 3: Security Scanning

```
flowchart TD
    A["push main"] --> G["security-scanners.yml"]
    B["pull_request to main"] --> G
    C["schedule (daily 03:47 UTC)"] --> G
    D["workflow_dispatch"] --> G

    G --> H["gitLeaks\n(secret detection)"]
    G --> I["semgrep\n(multi-language SAST)"]
    G --> J["grype\n(dependency SCA)"]
    G --> K["bandit\n(Python SAST)"]
    G --> L["checkov\n(IaC scanning)"]
    G --> M["clamav\n(malware scanning)"]

    H --> N["Upload SARIF\nto Code Scanning"]
    I --> N
    J --> N
    K --> N
    L --> N
    M --> O["Upload text log\n(artifact only)"]
```

All six scanner jobs run in parallel. Each scanner (except ClamAV) produces a SARIF report uploaded to both GitHub Code Scanning (Security tab) and as a downloadable workflow artifact. All scanners use a **deferred-failure pattern**: the scan runs to completion, results are always uploaded, and only then does the job fail if findings exceed the configured threshold. See the [Security Scanners Workflow](#) reference for details.

Pipeline 4: Pull Request Validation

```
flowchart TD
    A["pull_request_target\n(to main)"] --> B["get-pr-info"]
    C["merge_group\n(checks_requested)"] --> B

    B --> D["check-merge-status\n(HALT_MERGES + open release PRs)"]
    B --> E["fail-by-label\n(do-not-merge label)"]
    A --> F["validate\n(conventional commit title)"]
    A --> G["contributorStatement\n(acknowledgment in PR body)"]
    A --> H["auto-label\n(actions/labeler)"]
```

`pull-request-lint.yml` runs on every PR targeting `main` and on merge queue checks. It enforces four gates (conventional commit PR titles, the contributor statement from the PR template, a configurable merge-halt mechanism, and a do-not-merge label check) and automatically applies labels based on changed file paths. The workflow uses `pull_request_target` (not `pull_request`) so it runs in the context of the base branch — this is

safe because it never checks out PR code and the `auto-label` job uses `actions/labeler` which only reads file paths from the API.

Workflow Reference

Release PR Workflow (`release-pr.yml`)

Property	Value
File	<code>.github/workflows/release-pr.yml</code>
Trigger	<code>workflow_dispatch</code> with optional <code>version</code> input
Environment	<i>(none)</i>
Runner	<code>ubuntu-latest</code>

Purpose: Generates an updated `CHANGELOG.md` from conventional commits using git-cliff, writes the release version to `aidlc-rules/VERSION`, and opens a PR on a `release/vX.Y.Z` branch. This is the first step in the changelog-first release flow. The `aidlc-rules/VERSION` update ensures the PR touches `aidlc-rules/`, which triggers the `codebuild.yml` path filter and the `rules` auto-label.

Job: `release-pr` ("Create Release PR")

Step	Name	Action
1	Checkout code	<code>actions/checkout</code> with <code>fetch-depth: 0</code> (full history for git-cliff)
2	Install git-cliff	<code>orhun/git-cliff-action</code> to make the CLI available
3	Determine version	Use <code>inputs.version</code> (with semver validation) or <code>git-cliff --bumped-version</code> for auto-detection; falls back to patch bump from latest tag
4	Check tag does not exist	Fail early if the target tag already exists
5	Generate changelog	<code>orhun/git-cliff-action</code> with <code>--tag vX.Y.Z</code> to generate <code>CHANGELOG.md</code>
6	Create release PR	Write version to <code>aidlc-rules/VERSION</code> , check branch doesn't already exist, commit, push <code>release/vX.Y.Z</code> branch, open PR (with labels <code>release</code> and <code>rules</code> if they exist in the repo)

Version detection: If a version is specified, it must be valid semver (`MAJOR.MINOR.PATCH`); both `v0.2.0` and `0.2.0` are accepted. If no version is specified, `git-cliff --bumped-version` determines the next version from conventional commit prefixes. The `[bump]` config in `cliff.toml` controls the rules (e.g., `feat` → minor bump, breaking change → major bump). If no conventional commits are found, the workflow falls back to a patch bump from the latest tag. If no tags exist at all, it exits cleanly with a warning (no PR is created).

External actions (SHA-pinned):

Action	Version	SHA
<code>actions/checkout</code>	v6.0.1	<code>8e8c483db84b4bee98b60c0593521ed34d9990e8</code>
<code>orhun/git-cliff-action</code>	v4.7.0	<code>e16f179f0be49ecdfe63753837f20b9531642772</code>

Tag Release Workflow (`tag-on-merge.yml`)

Property	Value
File	<code>.github/workflows/tag-on-merge.yml</code>
Trigger	<code>pull_request: types: [closed]</code>
Condition	PR was merged AND branch name starts with <code>release/v</code>
Environment	<i>(none)</i>
Runner	<code>ubuntu-latest</code>

Purpose: Automatically creates a version tag on the merge commit when a release PR is merged, then dispatches `release.yml` (waits for completion) followed by `codebuild.yml`.

Job: `tag` ("Create Release Tag")

Step	Name	Action
1	Create tag	Extract version from branch name, verify tag doesn't exist, create via GitHub API
2	Dispatch release workflow and wait	<code>gh workflow run release.yml --ref \$TAG --repo \$REPO</code> , then <code>gh run watch</code> until completion
3	Dispatch codebuild workflow	<code>gh workflow run codebuild.yml --ref \$TAG --repo \$REPO</code> (runs after draft release exists)

Tag creation: Uses `gh api repos/.../git/refs` to create a lightweight tag.

Workflow dispatch: Tags created with `GITHUB_TOKEN` do not trigger `on: push: tags` events in other workflows. To work around this, `tag-on-merge.yml` explicitly dispatches `release.yml` and `codebuild.yml` via `gh workflow run --ref $TAG`. The `workflow_dispatch` event is exempt from this `GITHUB_TOKEN` limitation. Since `--ref` is set to the tag, both dispatched workflows see `github.ref = refs/tags/vX.Y.Z` — identical to a real tag push. The dispatches are **sequential**: `release.yml` runs first (watched via `gh run watch`) to ensure the draft release exists before `codebuild.yml` attempts to upload artifacts. If the release run cannot be found or fails, `codebuild.yml` is dispatched anyway as a fallback.

Security: The branch name `release/vX.Y.Z` is passed through an environment variable (not directly interpolated) to prevent command injection. The job-level `if` condition uses `github.event.pull_request.merged == true` to ensure only merged PRs trigger tagging.

CodeBuild Workflow (`codebuild.yml`)

Property	Value
File	<code>.github/workflows/codebuild.yml</code>
Triggers	<code>push</code> to <code>main</code> , <code>push</code> tags <code>v*</code> , <code>pull_request</code> to <code>main</code> (label-gated, path-filtered), <code>workflow_dispatch</code> (dispatched by <code>tag-on-merge.yml</code> or manual — select a tag in the UI to trigger a release build)
Environment	<code>codebuild</code> (protected, manual approval)
Runner	<code>ubuntu-latest</code>
Concurrency	Groups by <code>{workflow}-{event_name}-{ref}</code> , cancels in-progress

Purpose: Runs an AWS CodeBuild project, downloads primary and secondary artifacts from S3, caches them in GitHub Actions cache, uploads them as workflow artifacts, and (when triggered from a `v*` tag) attaches them to the GitHub Release.

PR label gate: For `pull_request` events, the workflow only fires when files under `aidlc-rules/**` are changed (via `paths` filter) and the `build` job only runs when the `rules` label is present on the PR (via `contains(github.event.pull_request.labels.*.name, 'rules')`). The `rules` label is applied automatically by the `auto-label` job in `pull-request-lint.yml` (see [Pull Request Validation Workflow](#)). The trigger includes `types: [opened, synchronize, reopened, labeled]` so that subsequent pushes to a labeled PR re-trigger the build automatically. `push`, `workflow_dispatch`, and tag events bypass the label check entirely.

Job: `label-reminder` (PR only, no `rules` label)

Step	Name	Action
1	Warn about missing rules label	Emits a <code>::warning::</code> annotation visible in the Actions summary
2	Comment on PR	Posts a one-time PR comment (idempotent — skips if the reminder comment already exists)

This job runs only for `pull_request` events where `aidlc-rules/**` changed but the `rules` label is absent. It alerts maintainers and reviewers that the evaluation pipeline was not triggered. The comment is posted once per PR using an HTML comment marker (`<!-- rules-label-reminder -->`) to avoid duplicates. In normal operation, the `auto-label` job in `pull-request-lint.yml` applies the `rules` label automatically, so this job serves as a fallback safety net.

Job: `label-cleanup` (PR only, `rules` label present)

Step	Name	Action
1	Remove label reminder comment	Finds and deletes the <code>label-reminder</code> PR comment (no-op if it doesn't exist)

This job runs when the `rules` label is applied, immediately removing the reminder comment without waiting for the `codebuild` environment approval gate.

Job: `build`

Step	Name	Condition	Action
1	List caches	(always)	<code>gh cache list</code> for existing project caches
2	Check cache	(always)	<code>actions/cache/restore</code> with <code>lookup-only: true</code>
3	Configure AWS credentials	cache miss	<code>aws-actions/configure-aws-credentials</code> (OIDC)
4	Run CodeBuild	cache miss	<code>aws-actions/aws-codebuild-run-build</code> with inline buildspec
5	Build ID	cache miss (always)	Echo CodeBuild build ID
6	Download CodeBuild artifacts	cache miss	Download primary + secondary artifacts from S3
7	List CodeBuild artifacts	cache miss	List and inspect downloaded zip files
8	Clean old report caches	cache miss	Delete 3 oldest matching caches for branch
9	Save report to cache	cache miss	<code>actions/cache/save</code> with key <code>{project}-{branch}-{sha}</code>
10	Upload primary artifact	<code>!env.ACT</code>	<code>actions/upload-artifact</code> for <code>{project}.zip</code>
11	Upload evaluation artifact	<code>!env.ACT</code>	<code>actions/upload-artifact</code> for <code>evaluation.zip</code>
12	Upload trend artifact	<code>!env.ACT</code>	<code>actions/upload-artifact</code> for <code>trend.zip</code>
13	Upload artifacts to release	triggered from a <code>v*</code> tag	Attach build artifacts to GitHub Release (draft or published)

Caching strategy: The cache key `{project}-{branch}-{sha}` ensures that the same commit on the same branch is never built twice. On cache hit, steps 3–9 are skipped entirely.

Inline buildspec: The workflow embeds a full `buildspec-override` rather than referencing an external file. The buildspec:

- Installs `gh` CLI (via `dnf`) and `uv` (Python package manager)
- Determines build context: release (tagged), pre-release (default branch), or pre-merge (feature branch)
- Creates placeholder evaluation and trend report files under `.codebuild/`
- Outputs a primary artifact (all files under `.codebuild/`) and two secondary artifacts (`evaluation`, `trend`)

Artifact upload compatibility: Upload steps are gated by `!env.ACT` because `actions/upload-artifact` v6 is incompatible with the `act` local runner.

External actions (all SHA-pinned):

Action	Version	SHA
<code>actions/cache/restore</code>	v5.0.3	<code>cdf6c1fa76f9f475f3d7449005a359c84ca0f306</code>
<code>aws-actions/configure-aws-credentials</code>	v6.0.0	<code>8df5847569e6427dd6c4fb1cf565c83acfa8afa7</code>
<code>aws-actions/aws-codebuild-run-build</code>	v1.0.18	<code>d8279f349f3b1b84e834c30e47c20dcb888b7e5</code>
<code>actions/cache/save</code>	v5.0.3	<code>cdf6c1fa76f9f475f3d7449005a359c84ca0f306</code>
<code>actions/upload-artifact</code>	v6.0.0	<code>b7c566a772e6b6bfb58ed0dc250532a479d7789f</code>

Release Workflow (`release.yml`)

Property	Value
File	<code>.github/workflows/release.yml</code>
Triggers	<code>workflow_dispatch</code> (dispatched by <code>tag-on-merge.yml</code>), <code>push</code> on tags matching <code>v*</code> (fallback for manual tag pushes)
Environment	<i>(none)</i>
Runner	<code>ubuntu-latest</code>

Purpose: Creates a **draft** GitHub Release with a zip of `aidlc-rules/` when dispatched or when a version tag is pushed. The release is kept as a draft so that CodeBuild artifacts can be attached and reviewed before publishing.

Job: `release` ("Create Release")

Step	Name	Condition	Action
1	Checkout code	<i>(always)</i>	<code>actions/checkout</code> with <code>fetch-depth: 0</code>
2	Extract version	<i>(always)</i>	Guard: if <code>GITHUB_REF</code> is not a <code>v*</code> tag, emit <code>::warning::</code> and skip remaining steps. Otherwise parse into <code>version</code> (no <code>v</code>) and <code>tag</code> (with <code>v</code>)
3	Create release artifact	ref is a <code>v*</code> tag	<code>zip -r ai-dlc-rules-v{VERSION}.zip aidlc-rules/</code>
4	Create GitHub Release	ref is a <code>v*</code> tag	<code>softprops/action-gh-release</code> with <code>draft: true</code> and zip attached

Graceful skip: If dispatched from a branch instead of a tag (e.g., someone manually runs the workflow from `main`), the job completes successfully with a warning annotation rather than failing. This prevents confusing red X failures in the Actions UI.

Release naming: `AI-DLC Workflow v{VERSION}` (e.g., `AI-DLC Workflow v0.1.6`)

External actions (SHA-pinned):

Action	Version	SHA
<code>actions/checkout</code>	v6.0.1	<code>8e8c483db84b4bee98b60c0593521ed34d9990e8</code>
<code>softprops/action-gh-release</code>	v2.5.0	<code>a06a81a03ee405af7f2048a818ed3f03bbf83c7b</code>

Pull Request Validation Workflow (`pull-request-lint.yml`)

Property	Value
File	<code>.github/workflows/pull-request-lint.yml</code>
Triggers	<code>pull_request_target</code> to <code>main</code> (edited, labeled, opened, ready_for_review, reopened, synchronize, unlabeled); <code>merge_group</code> (checks_requested)
Environment	<i>(none)</i>
Runner	<code>ubuntu-latest</code>
Concurrency	Groups by <code>{workflow}-{event_name}-{ref}</code> , cancels in-progress

Purpose: Validates pull requests before merge. Enforces conventional commit PR titles, the contributor acknowledgment statement, merge-halt controls, and a do-not-merge label gate. Also runs as a merge queue check.

Why `pull_request_target`: This trigger runs the workflow in the context of the base branch (not the PR head). This is safe here because no step checks out or executes PR code — the workflow only inspects PR metadata (title, labels, body). Using `pull_request_target` ensures the workflow has access to repository secrets and labels even for PRs from forks.

Job: `get-pr-info`

Step	Name	Action
1	Get PR info	Extract PR number and labels from event context (<code>pull_request_target</code>) or by API lookup (<code>merge_group</code>)

Outputs `pr_number` and `pr_labels` for downstream jobs. For `merge_group` events, the PR number is extracted from the ref name and labels are fetched via the GitHub API. For `pull_request_target` events, values come directly from the event payload.

Job: `check-merge-status` ("Check Merge Status")

Depends on `get-pr-info`. Runs `if: always()` so it executes even if the upstream job fails.

Check	Behavior
Open release PRs	Blocks merge if other <code>release/</code> PRs are open (prevents concurrent releases)
<code>HALT_MERGES = 0</code>	All merges allowed (default)
<code>HALT_MERGES = -N</code>	All merges blocked
<code>HALT_MERGES = N</code>	Only PR #N is allowed to merge

Job: `fail-by-label` ("Fail by Label")

Depends on `get-pr-info`. Runs `if: always()`. Fails the check if the PR has the `do-not-merge` label (configurable via `DO_NOT_MERGE_LABEL` variable).

Job: `validate` ("Validate PR title")

Only runs for `pull_request` and `pull_request_target` events (not `merge_group`). Uses `amannn/action-semantic-pull-request` to enforce conventional commit format on PR titles.

Allowed types: `fix`, `feat`, `build`, `chore`, `ci`, `docs`, `style`, `refactor`, `perf`, `test`. Scopes are optional (`requireScope: false`).

Job: `auto-label` ("Auto-label")

Only runs for `pull_request_target` events. Uses `actions/labeler` v6.0.1 to automatically apply and remove labels based on changed file paths. Label rules are defined in `.github/labeler.yml`:

Label	Path Pattern	Description
<code>rules</code>	<code>aidlc-rules/**</code>	Triggers CodeBuild evaluation pipeline
<code>documentation</code>	<code>**/*.md</code> (excluding <code>aidlc-rules/**</code>)	Non-rules markdown file changes
<code>github</code>	<code>.github/**</code>	Workflow, template, or config changes

With `sync-labels: true`, labels are automatically removed when the matching files are no longer in the PR diff (e.g., after a rebase drops those changes). New label rules can be added by editing `.github/labeler.yml` — no workflow changes required.

Job: `contributorStatement` ("Require Contributor Statement")

Only runs for `pull_request` and `pull_request_target` events. Skipped for bot accounts (`dependabot[bot]`, `github-actions[bot]`, `github-actions`, `aidlc-workflows`). Verifies the PR body contains the contributor acknowledgment text from `.github/pull_request_template.md`:

By submitting this pull request, I confirm that you can use, modify, copy, and redistribute this contribution, under the terms of the project license.

External actions (SHA-pinned):

Action	Version	SHA
<code>actions/labeler</code>	v6.0.1	<code>634933edcd8ababfe52f92936142cc22ac488b1b</code>
<code>amannn/action-semantic-pull-request</code>	v6.1.1	<code>48f256284bd46cdaab1048c3721360e808335d50</code>
<code>actions/github-script</code>	v8.0.0	<code>ed597411d8f924073f98dfc5c65a23a2325f34cd</code>

Security Scanners Workflow (`security-scanners.yml`)

Property	Value
File	<code>.github/workflows/security-scanners.yml</code>
Triggers	<code>push</code> to <code>main</code> , <code>pull_request</code> to <code>main</code> , <code>schedule</code> (daily 03:47 UTC), <code>workflow_dispatch</code>
Environment	<code>(none)</code>
Runner	<code>ubuntu-latest</code>
Concurrency	Groups by <code>{workflow}-{event_name}-{ref}</code> , cancels in-progress

Purpose: Runs six independent security scanners in parallel to detect secrets, vulnerabilities, misconfigurations, and malware. All HIGH and CRITICAL findings must be

remediated or have a documented risk acceptance before merge (see [Security Finding Requirements](#)).

Permissions model: Deny-all at workflow level, then each job grants only `actions: read`, `contents: read`, and `security-events: write`.

Jobs:

Job	Scanner	What it detects	Fails on
<code>gitleaks</code>	Gitleaks	Secrets in git history	Any secret not in <code>.gitleaks-baseline.json</code>
<code>semgrep</code>	Semgrep	Security anti-patterns (all languages)	Any finding (PRs: new findings only via <code>--baseline-commit</code>)
<code>grype</code>	Grype	Known CVEs in dependencies	High or critical CVEs (<code>fail-on-severity: high</code>)
<code>bandit</code>	Bandit	Python security issues	Any finding with high confidence
<code>checkov</code>	Checkov	IaC misconfigurations (GitHub Actions, Dockerfiles)	Any check failure (minus skipped checks)
<code>clamav</code>	ClamAV	Malware and viruses	Any detection

Deferred-failure pattern: All scanners capture the exit code without failing the step (`set +e`), upload the SARIF report as an artifact and to GitHub Code Scanning, then fail the job if findings were detected. This ensures results are always preserved regardless of outcome. ClamAV follows the same pattern but uploads a text log instead of SARIF.

Configuration files:

File	Purpose
<code>.bandit</code>	Bandit targets, excludes, confidence level
<code>.semgrepignore</code>	Semgrep path exclusions
<code>.gitleaks.toml</code>	Gitleaks ruleset extension and path allowlist
<code>.gitleaks-baseline.json</code>	Pre-existing known findings (test credentials)
<code>.grype.yaml</code>	Grype severity threshold and CVE ignore list
<code>.checkov.yaml</code>	Checkov frameworks and skipped checks

Version pinning: All scanner tool versions and GitHub Actions are pinned to specific versions or commit SHAs in the workflow file to ensure reproducible builds and prevent supply-chain attacks. These pins should be reviewed and updated periodically (at least quarterly). See [Updating Pinned Versions](#) for the update procedure.

For detailed remediation and suppression instructions, see [Developer's Guide — Security Scanners](#).

Protected Environments

Environment	Used By	Purpose
<code>codebuild</code>	<code>codebuild.yml</code> job <code>build</code>	Gates access to AWS credentials for CodeBuild

The `codebuild` environment is the only protected environment. It contains:

- The `AWS_CODEBUILD_ROLE_ARN` secret (required for OIDC-based AWS role assumption)
- Possibly the repository variables `CODEBUILD_PROJECT_NAME`, `AWS_REGION`, and `ROLE_DURATION_SECONDS` (these may alternatively be set at the repository level)

Environment protection rules (configured in GitHub repository settings) may include required reviewers or deployment branch restrictions.

Secrets and Variables

Secrets

Secret	Scope	Used By	Purpose
<code>AWS_CODEBUILD_ROLE_ARN</code>	Environment (<code>codebuild</code>)	<code>codebuild.yml</code>	IAM Role ARN for OIDC-based AWS STS role assumption
<code>GITHUB_TOKEN</code>	Automatic (GitHub- provided)	<code>release.yml</code> , <code>release-pr.yml</code> , <code>tag-on-merge.yml</code> , <code>pull-request- lint.yml</code>	Authenticate GitHub API calls (release creation, PR creation, tag creation, workflow dispatch, PR validation)

The `codebuild.yml` workflow also uses `github.token` (the automatic token, accessed without the `secrets.` prefix) for cache management and release asset uploads.

Repository Variables

Variable	Used By	Default Fallback	Purpose
<code>CODEBUILD_PROJECT_NAME</code>	<code>codebuild.yml</code>	<code>codebuild- project</code>	AWS CodeBuild project name
<code>AWS_REGION</code>	<code>codebuild.yml</code>	<code>us-east-1</code>	AWS region for CodeBuild and STS
<code>ROLE_DURATION_SECONDS</code>	<code>codebuild.yml</code>	<code>7200</code>	STS session duration (seconds)
<code>DO_NOT_MERGE_LABEL</code>	<code>pull-request- lint.yml</code>	<code>do-not-merge</code>	Label name that blocks PR merging
<code>HALT_MERGES</code>	<code>pull-request- lint.yml</code>	<code>0</code>	Merge gate: <code>0</code> = allow all, <code>-N</code> = block all, <code>N</code> = only PR #N

All variables have sensible defaults via `${{ vars.VAR || 'default' }}` syntax, so workflows run even without explicit variable configuration.

Permissions Model

Workflow-level permissions

Workflow	Permissions
<code>codebuild.yml</code>	All 16 scopes explicitly set to <code>none</code>
<code>pull-request-lint.yml</code>	All 16 scopes explicitly set to <code>none</code>
<code>release.yml</code>	All 16 scopes explicitly set to <code>none</code>
<code>release-pr.yml</code>	All 16 scopes explicitly set to <code>none</code>
<code>security-scanners.yml</code>	All 16 scopes explicitly set to <code>none</code>
<code>tag-on-merge.yml</code>	All 16 scopes explicitly set to <code>none</code>

Job-level permissions (overrides)

Workflow	Job	Permissions	Rationale
<code>codebuild.yml</code>	<code>label-reminder</code>	<code>pull-requests:</code> <code>write</code>	Post reminder comment when <code>rules</code> label is missing
<code>codebuild.yml</code>	<code>label-cleanup</code>	<code>pull-requests:</code> <code>write</code>	Delete reminder comment when <code>rules</code> label is applied
<code>codebuild.yml</code>	<code>build</code>	<code>actions: write</code> , <code>contents: write</code> , <code>id-token: write</code>	Cache management, release asset upload, OIDC token for AWS STS
<code>pull-request-lint.yml</code>	<code>auto-label</code>	<code>contents: read</code> , <code>issues: write</code> , <code>pull-requests:</code> <code>write</code>	Apply/remove labels based on changed file paths; <code>issues: write</code> allows creating labels that don't yet exist
<code>pull-request-lint.yml</code>	<code>get-pr-info</code>	<code>contents: read</code> , <code>pull-requests:</code> <code>read</code>	Read PR metadata and labels via API
<code>pull-request-lint.yml</code>	<code>check-merge-status</code>	<code>pull-requests:</code> <code>read</code>	Read PR state for merge gate checks
<code>pull-request-lint.yml</code>	<code>validate</code>	<code>pull-requests:</code> <code>read</code>	Read PR title for conventional commit validation
<code>pull-request-lint.yml</code>	<code>contributorStatement</code>	<code>pull-requests:</code> <code>read</code>	Read PR body for contributor acknowledgment
<code>release.yml</code>	<code>release</code>	<code>contents: write</code>	Create draft release and attach zip artifact

Workflow	Job	Permissions	Rationale
<code>release-pr.yml</code>	<code>release-pr</code>	<code>contents: write</code> , <code>pull-requests: write</code>	Generate changelog, push branch, open PR
<code>tag-on-merge.yml</code>	<code>tag</code>	<code>contents: write</code> , <code>actions: write</code>	Create tag via API, dispatch release and codebuild workflows

All six workflows follow a **deny-all-then-grant** pattern: every permission scope is set to `none` at the workflow level, then only the required scopes are granted at the job level. This is the strictest possible configuration and prevents privilege escalation from compromised steps. `security-scanners.yml` grants each of its six jobs `actions: read`, `contents: read`, and `security-events: write`.

Security Posture

Control	Implementation
Supply-chain protection	All external actions pinned to full commit SHAs (not mutable version tags)
AWS authentication	OIDC-based role assumption via <code>id-token: write</code> — no static credentials stored
Least-privilege tokens	All six workflows explicitly deny all 16 permission scopes at workflow level, grant only required scopes at job level
Environment protection	<code>codebuild</code> environment gates AWS credential access with potential reviewer/branch rules
Security scanning	Six automated scanners (SAST, SCA, secrets, IaC, malware) run on every push to <code>main</code> , every PR, and daily. Findings are published to GitHub Code Scanning. All HIGH and CRITICAL findings require remediation or documented risk acceptance
Label-gated CI	<code>codebuild.yml</code> requires the <code>rules</code> label on PRs and only triggers for <code>aidlc-rules/**</code> changes, preventing unnecessary builds and environment approval prompts. The label is applied automatically by the <code>auto-label</code> job in <code>pull-request-lint.yml</code>
Concurrency control	<code>codebuild.yml</code> , <code>pull-request-lint.yml</code> , and <code>security-scanners.yml</code> cancel in-progress runs for the same branch
Safe PR trigger	<code>pull-request-lint.yml</code> uses <code>pull_request_target</code> but never checks out PR code — only inspects metadata (title, labels, body)
Injection-safe inputs	Zero <code>\${{ }}</code> expression interpolation in <code>run:</code> blocks — all dynamic values (<code>github.ref_name</code> , <code>github.repository</code> , <code>env.*</code> , event inputs) passed via step-level <code>env:</code> or auto-exported workflow <code>env:</code> variables
Code ownership	<code>.github/</code> (including workflows) owned exclusively by <code>@awslabs/aidlc-admins</code> via CODEOWNERS
Account masking	<code>mask-aws-account-id: true</code> in AWS credential configuration

Security Finding Requirements

All **HIGH** and **CRITICAL** security findings from any scanner must be either **remediated** or have a **documented risk acceptance** before a PR can be merged to `main`. This applies to:

- **Bandit / Semgrep (SAST):** High-severity code findings must be fixed or suppressed with an inline comment (`# nosec` / `# nosemgrep`) that includes a justification explaining why the finding is acceptable
- **Grype (SCA):** High and critical CVEs must be resolved by upgrading the affected dependency. If no fix is available, add an entry to `.grype.yaml` `ignore` with the CVE, affected package, and reason for acceptance
- **Gitleaks (Secrets):** Any detected secret must be rotated immediately. Only synthetic/test credentials may be added to the baseline (`.gitleaks-baseline.json`)
- **Checkov (IaC):** Failing checks must be fixed or suppressed with an inline `# checkov:skip=` comment with a reason, or added to `.checkov.yaml` `skip-check` with a comment
- **ClamAV (Malware):** Any detection must be investigated and the file removed. No suppression mechanism exists

Risk acceptance process:

1. The developer adds the appropriate suppression (inline comment or config entry) with a clear justification
2. The suppression is reviewed as part of the normal PR code review process
3. Reviewers from `@awslabs/aidlc-admins` or `@awslabs/aidlc-maintainers` must approve any risk acceptance
4. LOW and MEDIUM findings should be addressed when practical but do not block merge

For detailed remediation and suppression instructions per scanner, see [Developer's Guide — Security Scanners](#).

Code Ownership

Defined in `.github/CODEOWNERS`:

Path	Owners
* (default)	@awslabs/aidlc-admins @awslabs/aidlc-maintainers
.github/	@awslabs/aidlc-admins
.github/CODEOWNERS	@awslabs/aidlc-admins
aidlc-rules/	@awslabs/aidlc-admins @awslabs/aidlc-maintainers @awslabs/aidlc-writers
assets/	@awslabs/aidlc-admins @awslabs/aidlc-maintainers @awslabs/aidlc-writers
scripts/	@awslabs/aidlc-admins @awslabs/aidlc-maintainers
CHANGELOG.md, cliff.toml, LICENSE, etc.	@awslabs/aidlc-admins

Key implication: Only @awslabs/aidlc-admins can approve changes to .github/ (workflows, CODEOWNERS, issue templates).

Release Process

Releases follow a **changelog-first** flow: the CHANGELOG is updated *before* the tag is created, so the tagged commit always contains its own changelog entry. The process has three human touchpoints (merge PR, approve CodeBuild, publish release).

1. **Dispatch the Release PR workflow** via the GitHub Actions UI:
2. Navigate to Actions → Release PR → Run workflow
3. Optionally specify a version (e.g., 0.2.0); leave blank to auto-determine from conventional commits
4. release-pr.yml generates CHANGELOG.md, writes the version to aidlc-rules/VERSION, and opens a PR on branch release/v1.2.0 with labels release and rules
5. **Review and merge the release PR:**
6. Verify the changelog content is correct
7. Merge the PR (requires @awslabs/aidlc-admins approval since CHANGELOG.md is owned by them)

8. `tag-on-merge.yml` automatically creates tag `v1.2.0` on the merge commit and dispatches the release and build workflows
9. `release.yml` **runs automatically** (dispatched by `tag-on-merge.yml` with `--ref v1.2.0`):
10. Zips `aidlc-rules/` into `ai-dlc-rules-v1.2.0.zip`
11. Creates a **draft** GitHub Release named "AI-DLC Workflow v1.2.0" with the zip attached
12. `codebuild.yml` **runs automatically** (dispatched by `tag-on-merge.yml`; requires `codebuild` environment approval):
13. Runs CodeBuild on the tagged commit
14. Downloads build artifacts (primary, evaluation, trend)
15. Attaches artifacts to the draft release (or creates a draft if one doesn't exist yet)
16. **Publish the release** by clicking "Publish release" in the GitHub UI:
17. Verify all expected artifacts are attached (rules zip + build artifacts)
18. Review release notes and edit if needed

Note: The `codebuild` protected environment may need its deployment branch rules updated to allow `v*` tags (in addition to `main`) for tag-triggered builds to proceed.

Changelog Configuration

Defined in `cliff.toml` (used by `release-pr.yml`):

Setting	Value
Commit format	Conventional commits (<code>feat:</code> , <code>fix:</code> , <code>docs:</code> , etc.)
Tag pattern	<code>v[0-9].*</code>
Sort order	Oldest first

Commit groups:

Prefix	Group Name
<code>feat</code>	Features
<code>fix</code>	Bug Fixes
<code>docs</code>	Documentation
<code>perf</code>	Performance
<code>refactor</code>	Refactoring
<code>style</code>	Style
<code>test</code>	Tests
<code>build</code>	CI/CD
<code>ci</code>	CI/CD
<code>chore</code>	Miscellaneous

Filtered commits:

Pattern	Action
<code>docs: update changelog</code>	Skipped (noise from previous release flow)

Unconventional commits are filtered out (`filter_unconventional = true`).

Version bump rules (defined in `[bump]` section):

Rule	Effect
<code>features_always_bump_minor = true</code>	<code>feat:</code> commits trigger a minor version bump
<code>breaking_always_bump_major = true</code>	Breaking changes trigger a major version bump

These rules are used by `git-cliff --bumped-version` when auto-determining the next version in `release-pr.yml`.

Updating Pinned Versions

All scanner tools, GitHub Actions, and container images in the workflow files are pinned to specific versions or commit SHAs. This prevents supply-chain attacks and ensures reproducible builds, but requires periodic maintenance to stay current with security patches and new features.

Pinned versions should be reviewed and updated **at least quarterly**.

Agent pre-commit checklist (recommended):

- `npx markdownlint-cli2 --fix "**/*.md" # auto-fix markdown lint issues`
- `npx markdownlint-cli2 "**/*.md" # verify no lint errors`
- `uv run pytest # run tests via uv wrapper`

Agents must run the checklist above and ensure all checks pass before committing and pushing changes.